U24AI006

ASSIGNMENT -09

1. Implement the Min-Max algorithm to solve a simple two-player Tic-Tac-Toe evaluate its performance, and visualize the search tree.
2. Extend Lab 1 with Alpha-Beta pruning, compare efficiency gains, and analyze pruning patterns on varied game trees.

CODE(1):

```
import sys
import pygame
import numpy as np
import random
import time

# tracking variables
total_nodes = 0
depth_limit = 0

# tree visualization settings
TREE_DISPLAY_DEPTH = 3  # how many levels of the tree to print

pygame.init()

# color palette
BG_COLOR = (28, 40, 51)
LINE_COLOR = (52, 73, 94)
CIRCLE_COLOR = (46, 204, 113)
CROSS_COLOR = (231, 76, 60)
TEXT_COLOR = (236, 240, 241)

# screen dimensions
SCREEN_W = 360
SCREEN_H = 360
LINE_W = 6
ROWS = 3
COLS = 3
CELL_SIZE = SCREEN_W // COLS
CIRC_RADIUS = CELL_SIZE // 3
CIRC_WIDTH = 12
X_WIDTH = 20

screen = pygame.display.set_mode((SCREEN_W, SCREEN_H))
pygame.display.set_caption('TicTacToe - Minimax')
```

```
screen.fill(BG_COLOR)

grid = np.zeros((ROWS, COLS))


def render_grid(color=LINE_COLOR):
    for i in range(1, ROWS):
        pygame.draw.line(screen, color, (0, i * CELL_SIZE), (SCREEN_W, i *
CELL_SIZE), LINE_W)
        pygame.draw.line(screen, color, (i * CELL_SIZE, 0), (i * CELL_SIZE,
SCREEN_H), LINE_W)


def render_marks():
    for r in range(ROWS):
        for c in range(COLS):
            cx = int(c * CELL_SIZE + CELL_SIZE // 2)
            cy = int(r * CELL_SIZE + CELL_SIZE // 2)
            if grid[r][c] == 1:
                pygame.draw.circle(screen, CIRCLE_COLOR, (cx, cy), CIRC_RADIUS,
CIRC_WIDTH)
            elif grid[r][c] == 2:
                offset = X_WIDTH
                pygame.draw.line(screen, CROSS_COLOR,
                                 (c * CELL_SIZE + offset, r * CELL_SIZE + offset),
                                 (c * CELL_SIZE + CELL_SIZE - offset, r * CELL_SIZE
+ CELL_SIZE - offset), X_WIDTH)
                pygame.draw.line(screen, CROSS_COLOR,
                                 (c * CELL_SIZE + offset, r * CELL_SIZE + CELL_SIZE
- offset),
                                 (c * CELL_SIZE + CELL_SIZE - offset, r * CELL_SIZE
+ offset), X_WIDTH)


def is_free(r, c):
    return grid[r][c] == 0


def place(r, c, player):
    grid[r][c] = player


def clear_cell(r, c):
    grid[r][c] = 0
```

```python
def board_full(b=grid):
    for r in range(ROWS):
        for c in range(COLS):
            if b[r][c] == 0:
                return False
    return True


def has_won(player, b=grid):
    for c in range(COLS):
        if b[0][c] == player and b[1][c] == player and b[2][c] == player:
            return True
    for r in range(ROWS):
        if b[r][0] == player and b[r][1] == player and b[r][2] == player:
            return True
    if b[0][0] == player and b[1][1] == player and b[2][2] == player:
        return True
    if b[0][2] == player and b[1][1] == player and b[2][0] == player:
        return True
    return False


def minimax(b, depth, maximizing, tree_node=None):
    """Minimax with optional tree recording for visualization."""
    global total_nodes, depth_limit
    total_nodes += 1
    depth_limit = max(depth_limit, depth)

    if has_won(2, b):
        if tree_node is not None:
            tree_node["score"] = float('inf')
            tree_node["label"] = "AI Win"
        return float('inf')
    if has_won(1, b):
        if tree_node is not None:
            tree_node["score"] = float('-inf')
            tree_node["label"] = "Human Win"
        return float('-inf')
    if board_full(b):
        if tree_node is not None:
            tree_node["score"] = 0
            tree_node["label"] = "Draw"
        return 0
```

```
    if maximizing:
        best = float('-inf')
        for r in range(ROWS):
            for c in range(COLS):
                if is_free(r, c):
                    place(r, c, 2)
                    # build child node if within display depth
                    child_node = None
                    if tree_node is not None and depth < TREE_DISPLAY_DEPTH:
                        child_node = {"move": (r, c), "player": "AI", "type": "MAX",
"children": []}
                        tree_node["children"].append(child_node)
                    val = minimax(b, depth + 1, False, child_node)
                    clear_cell(r, c)
                    best = max(best, val)
        if tree_node is not None:
            tree_node["score"] = best
        return best
    else:
        worst = float('inf')
        for r in range(ROWS):
            for c in range(COLS):
                if is_free(r, c):
                    place(r, c, 1)
                    child_node = None
                    if tree_node is not None and depth < TREE_DISPLAY_DEPTH:
                        child_node = {"move": (r, c), "player": "Human", "type":
"MIN", "children": []}
                        tree_node["children"].append(child_node)
                    val = minimax(b, depth + 1, True, child_node)
                    clear_cell(r, c)
                    worst = min(worst, val)
        if tree_node is not None:
            tree_node["score"] = worst
        return worst


def format_score(s):
    if s == float('inf'):
        return "+∞"
    elif s == float('-inf'):
        return "-∞"
    else:
        return str(int(s))
```

```
def print_tree(node, indent=0, is_last=True, prefix=""):
    """Print a tree node and its children as a visual text tree."""
    connector = "└── " if is_last else "├── "
    if indent == 0:
        line = prefix
    else:
        line = prefix + connector

    move_str = f"({node['move'][0]},{node['move'][1]})" if "move" in node else
"ROOT"
    player_str = node.get("player", "")
    node_type = node.get("type", "")
    score_str = format_score(node.get("score", 0))
    label = node.get("label", "")

    if label:
        line += f"{move_str} [{node_type}] → {score_str} ({label})"
    else:
        children_count = len(node.get("children", []))
        if children_count > 0:
            line += f"{move_str} [{node_type}] → {score_str}  ({children_count}
children)"
        else:
            line += f"{move_str} [{node_type}] → {score_str}  (pruned/leaf below
depth {TREE_DISPLAY_DEPTH})"

    print(line)

    children = node.get("children", [])
    for i, child in enumerate(children):
        is_child_last = (i == len(children) - 1)
        if indent == 0:
            child_prefix = prefix
        else:
            child_prefix = prefix + ("    " if is_last else "│   ")
        print_tree(child, indent + 1, is_child_last, child_prefix)


def compute_best_move():
    global depth_limit, total_nodes
    depth_limit = 0
    total_nodes = 0

    t_start = time.time()
```

```
    top_score = float('-inf')
    best = (-1, -1)

    # root of the visual tree
    root = {"type": "MAX", "player": "AI", "children": [], "move": None}

    for r in range(ROWS):
        for c in range(COLS):
            if is_free(r, c):
                place(r, c, 2)
                # create a child node for each top-level move
                child_node = {"move": (r, c), "player": "AI", "type": "MAX",
"children": []}
                root["children"].append(child_node)
                s = minimax(grid, 0, False, child_node)
                clear_cell(r, c)
                if s > top_score:
                    top_score = s
                    best = (r, c)

    elapsed = time.time() - t_start
    root["score"] = top_score
    print(f"\nMinimax: Nodes explored = {total_nodes}, Max depth = {depth_limit},
Time = {elapsed:.4f}s")

    # print the search tree (limited to TREE_DISPLAY_DEPTH levels)
    print(f"\n--- Minimax Search Tree (depth ≤ {TREE_DISPLAY_DEPTH}) ---")
    print(f"ROOT [MAX] → best score = {format_score(top_score)}, best move =
{best}")
    for i, child in enumerate(root["children"]):
        is_last = (i == len(root["children"]) - 1)
        print_tree(child, indent=1, is_last=is_last, prefix="")

    if best != (-1, -1):
        mark = "★" if top_score == float('inf') else "●"
        print(f"\n{mark} AI plays at {best}\n")
        place(best[0], best[1], 2)
        return True
    return False


def random_ai_move():
    t_start = time.time()
    open_cells = []
    for r in range(ROWS):
```

```
        for c in range(COLS):
            if is_free(r, c):
                open_cells.append((r, c))

    if open_cells:
        pick = random.choice(open_cells)
        place(pick[0], pick[1], 2)
        elapsed = time.time() - t_start
        print(f"Random Move: Time taken = {elapsed:.4f} seconds | Nodes explored = 1
| Max depth reached = 1")
        return True
    return False


def ai_move(level):
    roll = random.random()

    if level == 'easy':
        if roll <= 0.3:
            return compute_best_move()
        else:
            return random_ai_move()

    elif level == 'medium':
        if roll <= 0.6:
            return compute_best_move()
        else:
            return random_ai_move()

    elif level == 'impossible':
        return compute_best_move()


def reset():
    screen.fill(BG_COLOR)
    render_grid()
    for r in range(ROWS):
        for c in range(COLS):
            grid[r][c] = 0


def game_loop():
    render_grid()
    human_turn = True
    finished = False
```

```
    difficulty = 'impossible'

    while True:
        for ev in pygame.event.get():
            if ev.type == pygame.QUIT:
                pygame.quit()
                sys.exit()
            if ev.type == pygame.MOUSEBUTTONDOWN and not finished and human_turn:
                mx, my = ev.pos
                row_click = my // CELL_SIZE
                col_click = mx // CELL_SIZE
                if is_free(row_click, col_click):
                    place(row_click, col_click, 1)
                    render_marks()
                    if has_won(1):
                        print("Player wins!")
                        finished = True
                    elif board_full():
                        print("It's a tie!")
                        finished = True
                    else:
                        human_turn = False

        if not human_turn and not finished:
            if ai_move(difficulty):
                render_marks()
                if has_won(2):
                    print("AI wins!")
                    finished = True
                elif board_full():
                    print("It's a tie!")
                    finished = True
            human_turn = True
        pygame.display.update()

        if finished:
            pygame.time.wait(1000)
            reset()
            human_turn = True
            finished = False


game_loop()
```

CODE(2):

```
import sys
import pygame
import numpy as np
import random
import time

# tracking variables
total_nodes = 0
depth_limit = 0

pygame.init()

# color palette
BG_COLOR = (28, 40, 51)
LINE_COLOR = (52, 73, 94)
CIRCLE_COLOR = (46, 204, 113)
CROSS_COLOR = (231, 76, 60)
TEXT_COLOR = (236, 240, 241)

# screen dimensions
SCREEN_W = 360
SCREEN_H = 360
LINE_W = 6
ROWS = 3
COLS = 3
CELL_SIZE = SCREEN_W // COLS
CIRC_RADIUS = CELL_SIZE // 3
CIRC_WIDTH = 12
X_WIDTH = 20

screen = pygame.display.set_mode((SCREEN_W, SCREEN_H))
pygame.display.set_caption('TicTacToe - Alpha Beta')
screen.fill(BG_COLOR)

grid = np.zeros((ROWS, COLS))


def render_grid(color=LINE_COLOR):
   # draw horizontal and vertical grid lines
   for i in range(1, ROWS):
       pygame.draw.line(screen, color, (0, i * CELL_SIZE), (SCREEN_W, i *
CELL_SIZE), LINE_W)
       pygame.draw.line(screen, color, (i * CELL_SIZE, 0), (i * CELL_SIZE,
SCREEN_H), LINE_W)
```

```
def render_marks():
    for r in range(ROWS):
        for c in range(COLS):
            cx = int(c * CELL_SIZE + CELL_SIZE // 2)
            cy = int(r * CELL_SIZE + CELL_SIZE // 2)
            if grid[r][c] == 1:
                pygame.draw.circle(screen, CIRCLE_COLOR, (cx, cy), CIRC_RADIUS,
CIRC_WIDTH)
            elif grid[r][c] == 2:
                offset = X_WIDTH
                pygame.draw.line(screen, CROSS_COLOR,
                                 (c * CELL_SIZE + offset, r * CELL_SIZE + offset),
                                 (c * CELL_SIZE + CELL_SIZE - offset, r * CELL_SIZE
+ CELL_SIZE - offset), X_WIDTH)
                pygame.draw.line(screen, CROSS_COLOR,
                                 (c * CELL_SIZE + offset, r * CELL_SIZE + CELL_SIZE
- offset),
                                 (c * CELL_SIZE + CELL_SIZE - offset, r * CELL_SIZE
+ offset), X_WIDTH)


def is_free(r, c):
    return grid[r][c] == 0


def place(r, c, player):
    grid[r][c] = player


def clear_cell(r, c):
    grid[r][c] = 0


def board_full(b=grid):
    for r in range(ROWS):
        for c in range(COLS):
            if b[r][c] == 0:
                return False
    return True


def has_won(player, b=grid):
    # check columns
```

```
    for c in range(COLS):
        if b[0][c] == player and b[1][c] == player and b[2][c] == player:
            return True
    # check rows
    for r in range(ROWS):
        if b[r][0] == player and b[r][1] == player and b[r][2] == player:
            return True
    # diagonals
    if b[0][0] == player and b[1][1] == player and b[2][2] == player:
        return True
    if b[0][2] == player and b[1][1] == player and b[2][0] == player:
        return True
    return False


def minimax(b, depth, maximizing, alpha, beta):
    global total_nodes, depth_limit
    total_nodes += 1
    depth_limit = max(depth_limit, depth)

    if has_won(2, b):
        return float('inf')
    if has_won(1, b):
        return float('-inf')
    if board_full(b):
        return 0

    if maximizing:
        best = float('-inf')
        for r in range(ROWS):
            for c in range(COLS):
                if is_free(r, c):
                    place(r, c, 2)
                    val = minimax(b, depth + 1, False, alpha, beta)
                    alpha = max(alpha, val)
                    clear_cell(r, c)
                    best = max(best, val)
                    if beta <= alpha:
                        return best
        return best
    else:
        worst = float('inf')
        for r in range(ROWS):
            for c in range(COLS):
                if is_free(r, c):
```

```
                    place(r, c, 1)
                    val = minimax(b, depth + 1, True, alpha, beta)
                    beta = min(beta, val)
                    clear_cell(r, c)
                    worst = min(worst, val)
                    if beta <= alpha:
                        return worst
        return worst


def compute_best_move():
    alpha = float('-inf')
    beta = float('inf')

    global depth_limit, total_nodes
    depth_limit = 0
    total_nodes = 0

    t_start = time.time()
    top_score = float('-inf')
    best = (-1, -1)
    for r in range(ROWS):
        for c in range(COLS):
            if is_free(r, c):
                place(r, c, 2)
                s = minimax(grid, 0, False, alpha=alpha, beta=beta)
                clear_cell(r, c)
                if s > top_score:
                    alpha = max(alpha, s)
                    top_score = s
                    best = (r, c)
    elapsed = time.time() - t_start
    print(f"Minimax: Nodes explored = {total_nodes}, Max depth reached =
{depth_limit}, Time taken = {elapsed:.4f} seconds")
    if best != (-1, -1):
        place(best[0], best[1], 2)
        return True
    return False


def random_ai_move():
    t_start = time.time()
    open_cells = []
    for r in range(ROWS):
        for c in range(COLS):
```

```
            if is_free(r, c):
                open_cells.append((r, c))

    if open_cells:
        pick = random.choice(open_cells)
        place(pick[0], pick[1], 2)
        elapsed = time.time() - t_start
        print(f"Random Move: Time taken = {elapsed:.4f} seconds | Nodes explored = 1
| Max depth reached = 1")
        return True
    return False


def ai_move(level):
    roll = random.random()

    if level == 'easy':
        if roll <= 0.3:
            return compute_best_move()
        else:
            return random_ai_move()

    elif level == 'medium':
        if roll <= 0.6:
            return compute_best_move()
        else:
            return random_ai_move()

    elif level == 'impossible':
        return compute_best_move()


def reset():
    screen.fill(BG_COLOR)
    render_grid()
    for r in range(ROWS):
        for c in range(COLS):
            grid[r][c] = 0


def game_loop():
    render_grid()
    human_turn = True
    finished = False
```

```
    difficulty = 'impossible'

    while True:
        for ev in pygame.event.get():
            if ev.type == pygame.QUIT:
                pygame.quit()
                sys.exit()
            if ev.type == pygame.MOUSEBUTTONDOWN and not finished and human_turn:
                mx, my = ev.pos
                row_click = my // CELL_SIZE
                col_click = mx // CELL_SIZE
                if is_free(row_click, col_click):
                    place(row_click, col_click, 1)
                    render_marks()
                    if has_won(1):
                        print("Player wins!")
                        finished = True
                    elif board_full():
                        print("It's a tie!")
                        finished = True
                    else:
                        human_turn = False

        if not human_turn and not finished:
            if ai_move(difficulty):
                render_marks()
                if has_won(2):
                    print("AI wins!")
                    finished = True
                elif board_full():
                    print("It's a tie!")
                    finished = True
            human_turn = True
        pygame.display.update()

        if finished:
            pygame.time.wait(1000)
            reset()
            human_turn = True
            finished = False


game_loop()
```